

Product Support Agentic AI Agent with Contextual Memory and Multi-Document Reasoning

High-Level Design (HLD)

Title Page

Field	Value
Document Title	Product Support Agentic AI Agent with Contextual Memory and Multi-Document Reasoning - HLD
Document ID	HCL-PSAAI-HLD-001
Version	1.1
Date	2026-08-19
Author	Codex Documentation Review
Company	HCL
Source Document	Product_Support_Agentic_AI_Enterprise_Documentation.docx

Revision History

Version No	Date	Prepared by / Modified by	Significant Changes
1.0	2026-06-13	Chief Software Architect	Initial Software Architecture Document / High-Level Design
1.1	2026-08-19	Codex Documentation Review	Reviewed and aligned the HLD to the current repository architecture and runtime flow

Table of Contents

- 1. Overview
- 2. Architectural Goals
- 3. System Architecture
- 4. Document Ingestion Architecture
- 5. Retrieval and Grounded RAG Architecture
- 6. Contextual Query Rewriting and Memory
- 7. LLM Integration Architecture
- 8. Streamlit UI Architecture
- 9. Data and Storage Architecture
- 10. Deployment and Runtime Configuration
- 11. Security and Failure Handling

- 12. Performance and Scalability Notes
- 13. Risks and Trade-offs
- 14. Documentation Consistency Notes
- 15. References

1. Overview

This HLD describes how the current repository is architecturally organized. It focuses on subsystem boundaries, orchestration, data flow, storage layout, and design decisions. It does not attempt to define every class or method.

2. Architectural Goals

The implemented architecture prioritizes:

- grounded answers from retrieved evidence only
- local document ingestion and local vector storage
- simple workstation deployment
- clear modular service boundaries
- support for configurable LLM providers
- observable and testable control flow

3. System Architecture

3.1 Layered Architecture

The application is organized into these top-level layers:

Layer	Responsibility
Streamlit UI Layer	Renders ingestion, chat, sidebar metrics, and developer tools
Application / Orchestration Layer	Coordinates ingestion and grounded RAG execution
Retrieval Layer	Validates queries, embeds queries, retrieves and reranks chunks
Knowledge-Base Layer	Persists uploads, manifests, vector records, and FAISS artifacts
Provider Layer	Executes grounded generation through a selected LLM provider
Shared Core	Configuration, logging, models, exceptions, and utilities

3.2 Primary Architectural Components

Component	Responsibility
KnowledgeBaseIngestionPipeline	Upload-to-index workflow
UploadManager	Upload validation, checksum duplicate detection, local persistence
DocumentLoader	Delegates file-type-specific extraction
ChunkingService	Structure-aware chunk generation

Component	Responsibility
<code>EmbeddingService</code>	Local Sentence Transformers embeddings
<code>VectorStoreService</code>	Persistent FAISS creation, loading, updating, search
<code>RetrieverService</code>	Query validation, embedding, search, reranking, context expansion
<code>QueryContextualizer</code>	Resolves follow-up questions into standalone retrieval queries
<code>ConversationMemoryService</code>	Session-scoped in-memory conversation history
<code>RAGService</code>	LangGraph orchestration for grounded answer generation
<code>build_llm_service()</code>	Provider factory for Gemini or Ollama
<code>KnowledgeBaseService</code>	Clear knowledge-base lifecycle operation

4. Document Ingestion Architecture

4.1 Ingestion Flow

The ingestion path is:

`Streamlit Upload -> UploadManager -> DocumentLoader -> ChunkingService -> EmbeddingService -> VectorStoreService`

4.2 Upload and Deduplication

The current architecture performs duplicate control before saving:

- compute SHA-256 from uploaded bytes
- compare against the persistent upload manifest
- skip the upload if the same checksum already exists
- keep filename collision renaming only for same filename with different content

4.3 Extraction Strategy

The architecture uses separate loaders:

File Type	Loader Behavior
PDF	Page-by-page extraction with text normalization and table-row shaping
TXT	Whole-document text extraction
CSV	Row-level extraction with <code>column: value</code> text composition

4.4 Chunking Design

Chunking is not a raw fixed-split process only. The implemented architecture:

- builds semantic blocks
- preserves section titles when detected

- keeps table-like lines together when possible
- uses configurable `chunk_size` and `chunk_overlap`
- emits stable chunk IDs in `file:page:row:chunk` format

5. Retrieval and Grounded RAG Architecture

5.1 Retrieval Pipeline

The retrieval architecture is:

`Validated Query -> Local Query Embedding -> FAISS Candidate Search -> Reranking -> Deduplication -> Adjacent Chunk Expansion -> Relevance Filtering`

5.2 Retrieval Quality Controls

The current architecture includes:

- candidate pool expansion beyond the final top-k
- lexical term-overlap boosting
- section-title overlap boosting
- exact phrase boosting
- structural penalties for low-value chunks such as repeated chapter headers
- duplicate-result suppression
- adjacent chunk expansion for context completion

5.3 Grounded RAG Flow

The answer-generation path is:

`User Question -> QueryContextualizer -> RetrieverService -> RAGService -> LLM Provider -> Deterministic Citations`

The system does not allow the answering model to act as an open-domain assistant. If usable evidence is not available, the architecture returns a fallback response instead of a speculative answer.

5.4 Multi-Document Reasoning

When multiple source documents are relevant and the query indicates synthesis or comparison, the architecture uses:

1. document grouping
2. per-document evidence summarization
3. final synthesis from document evidence summaries
4. deterministic citation formatting from retrieval metadata

6. Contextual Query Rewriting and Memory

6.1 Conversation Memory

Conversation memory is:

- session-scoped
- in-memory
- stored through Streamlit session-backed storage
- limited to recent user and grounded assistant turns

6.2 Contextualization Architecture

The `QueryContextualizer` receives:

- current user query
- recent memory messages
- a dedicated contextualization prompt

If contextualization fails, returns an empty result, or the provider stops abnormally, the architecture falls back to the original user query.

7. LLM Integration Architecture

7.1 Provider Abstraction

The current codebase uses a provider abstraction:

- `BaseLLMService`
- `OllamaService`
- `GeminiLLMAdapter` wrapping `GeminiService`
- `build_llm_service(settings)`

7.2 Active Runtime Selection

Provider selection is resolved from environment configuration. The current local project environment is configured for:

- provider: `ollama`
- model: `qwen3:8b`
- endpoint: `http://localhost:11434`

Gemini support remains in the repository as an optional provider path.

7.3 Provider Responsibilities

The provider layer is responsible for:

- contextualize query
- generate grounded answer
- summarize document evidence
- synthesize multi-document answers

Provider-specific logic is kept outside the retriever and vector-store layers.

8. Streamlit UI Architecture

8.1 Primary Experiences

The current UI exposes two primary experiences:

- Ask AI
- Ingestion

8.2 Sidebar Responsibilities

The sidebar contains:

- workspace status
- indexed document and chunk counts
- provider and model status
- conversation turn count
- search-depth slider
- clear conversation action
- developer tools

8.3 Developer Tools

Developer tools are intentionally separated from the main chat flow and show:

- latest sources
- debug information
- retrieval details
- metadata snapshots

9. Data and Storage Architecture

9.1 Runtime Paths

Artifact	Path
uploads	data/uploads
FAISS directory	data/faiss
metadata directory	data/metadata
prompts	prompts
logs	logs/product_support_agent.log

9.2 Persistent Files

File	Purpose
data/faiss/index.faiss	FAISS vector index
data/faiss/index.pkl	vector state and chunk records
data/metadata/uploaded_files.json	persistent upload manifest

9.3 Metadata Model

The architecture preserves these metadata fields where available:

- file name
- document type
- page number
- row number
- chunk number
- chunk ID
- section title
- vector ID

10. Deployment and Runtime Configuration

The application is a local Python/Streamlit deployment. Configuration is loaded from:

- project root `.env`
- code defaults in `config.py`

The architecture does **not** use a YAML runtime configuration file in the current implementation.

11. Security and Failure Handling

11.1 Security Boundaries

- raw uploaded files remain local
- embeddings remain local
- FAISS index remains local
- model prompts receive constructed context, not filesystem paths
- API keys are environment-based

11.2 Fallback Behavior

The architecture uses safe fallbacks for:

- missing FAISS index
- no retrieval results
- low-relevance retrieval results
- provider configuration errors
- provider generation failures
- empty provider responses

The primary insufficient-evidence fallback is:

I could not find enough information in the available knowledge base to answer this question.

12. Performance and Scalability Notes

The repository implements local-first performance techniques such as:

- normalized local embeddings
- FAISS `IndexFlatIP`
- persisted index reuse after restart
- candidate-pool retrieval
- lightweight in-memory conversation state

Hard contractual performance benchmarks are not fully specified in code and are therefore not stated here as verified architecture guarantees.

13. Risks and Trade-offs

Topic	Trade-off
Local-first design	Strong privacy and simple deployment, but single-node scaling
Ollama local generation	Greater locality and privacy, but depends on local model availability and local inference speed
Session-only memory	Simpler and safer than durable memory, but no long-term conversation persistence
FAISS local persistence	Lightweight and fast, but not a distributed multi-user vector service
Grounded-only fallback	Safer than speculative answering, but may refuse some questions even when users expect a best-effort response

14. Documentation Consistency Notes

- This reviewed HLD reflects the current repository, where LangGraph orchestration is already active rather than planned for a future phase.
- The current implementation uses environment-based configuration, not YAML runtime configuration.
- The active local environment uses `ollama` with `qwen3:8b`, while Gemini remains an optional supported provider in code.
- Streamlit developer details are separated from the conversation rather than embedded after every assistant response.

15. References

1. [README.md](file:///D:/hcl/README.md)
2. [config.py](file:///D:/hcl/src/product_support_agent/config.py)
3. [rag_service.py](file:///D:/hcl/src/product_support_agent/services/rag_service.py)
4. [retriever.py](file:///D:/hcl/src/product_support_agent/services/retriever.py)
5. [indexing_pipeline.py](file:///D:/hcl/src/product_support_agent/services/indexing_pipeline.py)
6. [sections.py](file:///D:/hcl/src/product_support_agent/ui/sections.py)
7. [layout.py](file:///D:/hcl/src/product_support_agent/ui/layout.py)